
Social Media Event Organizer

(Group No. 17)

*Submitted in partial fulfilment of the requirements of
ECOM F213 Object Oriented Programming*

By

<u>No.</u>	<u>Member Name</u>	<u>ID Number</u>
1.	Harshit Rawat	2024AAPS0756G
2.	Abhishek Bhadani	2024A8PS0730G
3.	Shreyas Lele	2024A3PS0379G
4.	Yash Palod	2024AAPS0742G



BIRLA INSTITUTE OF TECHNOLOGY AND SCIENCE PILANI, GOA CAMPUS

April 2026

1. Package EventManagementSystem

A. EventComponent Interface

Methods:

- `public void display():` To be implemented in `BasicEvent` and `CompositeEvent`. Handles the rendering of the event.
- `public void add(EventComponent event):` To be implemented in `BasicEvent` and `CompositeEvent`. Designed to append a new child component into the event.
- `public void remove(EventComponent event):` To be implemented in `BasicEvent` and `CompositeEvent`. Designed to remove/delete an existing child component from the event.
- `public EventComponent getChild(int index):` To be implemented in `BasicEvent` and `CompositeEvent`. Used to access and retrieve a specific nested sub-event based on the index.

B. BasicEvent Class (Implements EventComponent)

Fields:

- `protected String name:` The title or name identifying the event.
- `protected String description:` A brief summary or explanation of the event's purpose.
- `protected String date:` The scheduled date for the event.
- `protected String time:` The scheduled start time for the event.
- `protected String location:` The venue or physical address where the event will take place.
- `protected String type:` The category or classification of the event (e.g., concert, meeting, party).

Methods:

- `public BasicEvent(String name, String description, String date, String time, String location, String type):` Constructor that initializes all the attributes of the basic event upon creation.
- `public void getInvited():` Initiates the invitation process. It does not handle the logic itself. It delegates the current event instance to the `InviteManagerFacade.getInvitedToEvent()` method.
- `public void display():` Handles the rendering of the event. It delegates the current instance to the `DashboardFacade.displayEvent()` method.

- `public void add(EventComponent event)`: Structural composite method. Since `BasicEvent` is a leaf, it throws an `UnsupportedOperationException` stating "BasicEvent cannot have children."
- `public void remove(EventComponent event)`: Structural composite method. Throws an `UnsupportedOperationException` stating "BasicEvent cannot remove children."
- `public EventComponent getChild(int index)`: Structural composite method. Throws an `UnsupportedOperationException` stating "BasicEvent has no children."

C. CompositeEvent Class (Extends BasicEvent, Implements EventComponent)

Fields:

- `private List<EventComponent> children`: A list storing all the sub-events or components contained within this composite event.

Methods:

- `public CompositeEvent(List<EventComponent> children)`: Constructor that initializes the composite event with a pre-existing list of child components. It calls the superclass constructor with empty strings for the basic event details.
- `public CompositeEvent(String name, String description, String date, String time, String location, String type)`: Overloaded Constructor that initializes the composite event with its core details by passing them to the superclass. It initializes an empty `ArrayList` for the children collection.
- `public void add(EventComponent event)`: Appends the specified `EventComponent` (which could be a `BasicEvent` or another `CompositeEvent`) to the children list.
- `public void remove(EventComponent event)`: Checks if the specified `EventComponent` exists in the children list and removes it if present.
- `public EventComponent getChild(int index)`: Retrieves and returns the sub-event located at the specified index within the children list.
- `protected List<EventComponent> getChildren()`: Returns the complete list of child components. This is accessible only within the package and by subclasses.

D. MusicEventDecorator Class (extends EventDecorator)

Fields:

- `private String BandName`: Stores the name of the musical band or artist performing at the decorated event.

Methods:

- `public MusicEventDecorator(EventComponent decoratedEvent, String BandName)`: Constructor that accepts an existing `EventComponent` to wrap (passing it to the

EventDecorator superclass) and sets the initial band name for this specific decoration.

- `public String getBandName():` Returns the name of the band associated with this event.
- `public void setBandName(String bandName):` Updates or assigns the name of the performing band for this event.

E. NewsEventDecorator Class (extends EventDecorator)

Fields:

- `private String headline:` Stores the primary news headline associated with the decorated event.
- `private String description:` Stores a secondary, more detailed news description or broadcast summary for the event.

Methods:

- `public NewsEventDecorator(EventComponent decoratedEvent, String headline, String description):` Constructor that accepts an existing EventComponent to wrap (passing it to the EventDecorator superclass).
- `public String getHeadline():` Retrieves the news headline associated with this event.
- `public void setHeadline(String headline):` Updates or assigns the primary news headline for this event.
- `public String getDescription():` Retrieves the secondary news description associated with this event.
- `public void setDescription(String description):` Updates or assigns the detailed news description for this event.

F. OutingEventDecorator Class (extends EventDecorator)

Fields:

- `private String hotel:` Stores the name of the hotel or accommodation associated with the outing event.
- `private int starRating:` Stores the numeric star rating (quality classification) of the hotel.

Methods:

- `public OutingEventDecorator(EventComponent decoratedEvent, int starRating, String hotel):` Constructor that accepts an existing EventComponent to wrap (passing it to

the EventDecorator superclass), and assigns the initial star rating and hotel name for this specific decoration.

- public int getStarRating(): Retrieves the star rating of the hotel associated with this event.
- public void setStarRating(int starRating): Updates or assigns the star rating of the hotel for this event.
- public String getHotel(): Retrieves the name of the hotel associated with this event.
- public void setHotel(String hotel): Updates or assigns the name of the hotel for this event.

G. UrgentEventDecorator Class (extends EventDecorator)

Fields:

- private int UrgencyLevel: A numeric value representing the priority or severity of the event (e.g., 1 for low, 5 for critical).

Methods:

- public UrgentEventDecorator(EventComponent decoratedEvent, int UrgencyLevel): Constructor that accepts an existing EventComponent to wrap (passing it to the EventDecorator superclass), and initializes the specific urgency level for this decoration.
- public int getUrgencyLevel(): Retrieves the current urgency level assigned to this event.
- public void setUrgencyLevel(int urgencyLevel): Updates or assigns the numeric urgency level for this event.

H. EventMangementFacade Class

J. MainEventManager Class

Methods:

- public static void createBasicEvent(): Opens a Scanner to sequentially prompt the user for the core attributes of a new event (name, description, date, time, location, and type). *(Note: Based on the current code, this method is partially complete as it collects the input variables but does not yet instantiate the BasicEvent object or save it to a repository).*

2. Package Friend

A. FriendManagement (Interface)

B. Follower Class (Implements FriendManagement)

Fields:

- int userid: The unique identifier of the user who is being followed.
- int followerid: The unique identifier of the user who is doing the following.
- String status: A string literal representing the relationship state (initialized as "FOLLOWER").
- LocalDateTime establishedDateTime: The exact timestamp when this relationship was created.

Methods:

- public Follower(int userid, int followerid): Constructor that initializes the user and follower IDs, gives the status of "FOLLOWER" , and automatically captures the current system time for establishedDateTime.

C. Following Class (Implements FriendManagement)

Fields:

- int userid: The unique identifier of the user who is executing the follow action.
- int followingid: The unique identifier of the target user being followed.
- String status: A string literal representing the relationship state (initialized as "FOLLOWING").
- LocalDateTime establishedDateTime: The exact timestamp when this relationship was created.

Methods:

- public Following(int userid, int followingid): Constructor that initializes the IDs, hardcodes the status to "FOLLOWING", and automatically sets the creation timestamp.

D. Friend Class (Implements FriendManagement)

Fields:

- int userid: The unique identifier of the first user in the friendship.
- int friendid: The unique identifier of the second user in the friendship.
- String status: A string literal representing the relationship state (initialized as "FRIEND").
- int closeness: A numeric metric representing the strength, tier, or closeness level of the friendship.

Methods:

- `public Friend(int userid, int friendid, int c)`: Constructor that establishes a mutual friendship by initializing both user IDs and their associated closeness metric.

E. FriendRepo Class

Fields:

- `private static Path friendFilePath`: The system path to the text file storing mutual friendship records.
- `private final Path followFilePath`: The system path to the text file storing unidirectional follower/following records.

Methods:

- `public FriendRepo(String friend_filePath, String follow_filePath)`: Constructor that initializes the file paths and immediately calls helper methods to ensure both files exist on the disk.
- `public void createFileIfNotExists_friend()` / `public void createFileIfNotExists_follow()`: Utility methods that safely check for and create the necessary directories and text files if they are missing.
- `public void createRelationship(User user, User targetuser, int closeness)`: Overloaded method for mutual friendships. Formats the user IDs and closeness score into a CSV string and appends it to the friend file.
- `public void createRelationship(User user, User targetuser)`: Overloaded method for unidirectional follows. Appends the user IDs to the storage file.
- `public List<FriendManagement> getRelationship(User u, String type)`: The core retrieval engine. Reads the specific relationship file, parses the CSV lines, filters for the requested user's ID, and maps the raw data into Friend, Follower, or Following objects based on the requested type.
- `public void deleteRelationship(int id1, int id2)`: Orchestrates the removal of a relationship between two users by triggering the rewrite process on both the friend and follow files.
- `private static void processAndRewrite(Path filePath, int targetId1, int targetId2)`: A complex utility method that loads a file into memory, strips out any line matching the two target IDs (effectively deleting the relationship), and completely overwrites the physical file with the surviving records.
- `public List<FriendManagement> getAllRelationship(User u)` / `public List<FriendManagement> getAllRelationship(User u, String type)`: Overloaded convenience methods that aggregate and return a user's relationships, optionally filtered by type.

- `public static void createFriend(String s)`: A static helper method specifically used by the Invite SubSystem to instantly create a mutual friendship record from a raw invite CSV string (appending a default closeness score of 0).

3. Package UserMangerSubSystem

A. AppStartup Class

Fields:

- `private static AppStartup instance`: The single, static instance of the AppStartup class.
- `private ConfigLoader configLoader`: Utility for loading application settings from a properties file.
- `private SystemDataService systemDataService`: Service responsible for loading initial background system data.
- `private UserRepository userRepository`: The repository handling persistent user data storage and retrieval.

Methods:

- `private AppStartup()`: Private constructor that initializes the loaders and repositories, fetching the specific file paths from the configuration.
- `public static synchronized AppStartup getInstance()`: Returns the singleton instance of the application startup class, creating it if it does not yet exist.
- `public void startApplication()`: The core execution method. It loads configurations, simulates a startup delay if operating in "dev" mode, and drops the user into an infinite while-loop that prompts the UserLogin or UserRegistration flows via console input.

B. ConfigLoader Class

Fields:

- `private final Properties properties`: A Java Properties object that stores the key-value pairs loaded from the external configuration file.

Methods:

- `public ConfigLoader()`: Constructor that attempts to open and load the `config.properties` file into memory. Throws a `RuntimeException` if the file cannot be found or read.
- `public String getProperty(String key)`: Retrieves the string value associated with the specified key. Returns null if the key is not found.
- `public String getProperty(String key, String defaultValue)`: Retrieves the string value for a key, but returns the provided `defaultValue` if the key does not exist.

- `public int getIntProperty(String key, int defaultValue)`: Safely parses and returns an integer property. Returns the `defaultValue` if the key is missing or contains non-numeric text.
- `public boolean getBooleanProperty(String key, boolean defaultValue)`: Safely parses and returns a boolean property. Returns the `defaultValue` if the key is missing.
- `public String loadConfig()`:

C. PassWordManager Class

Constants:

- `private static final String ALGORITHM`: Defines the encryption standard used (value: "AES").
- `private static final String SECRET_KEY`: The 16-character symmetric key required for AES-128 encryption and decryption.

Fields:

- `private final Path passWordfilepath`: The designated file path used for storing the encrypted passwords in a comma-separated format.

Methods:

- `public PassWordManager(String filePath)`: Constructor that initializes the file path and calls the helper to ensure the file exists.
- `public void createFileIfNotExists()`: Helper method that safely checks for and creates the directory and file for password records if they do not already exist.
- `private static SecretKeySpec getKey()`: A private helper method that converts the string `SECRET_KEY` into a standardized `SecretKeySpec` object for the cipher.
- `public static String encrypt(String plainText)`: Takes a raw string, encrypts it using the AES cipher, and returns a safely encoded Base64 string.
- `public static String decrypt(String encryptedText)`: Takes a Base64 encrypted string, decodes it, and decrypts it back to the original readable text.
- `protected String getPassWord(int id)`: Searches the text file for a specific user ID, decrypts their associated password, and returns the plain text string.
- `protected void setPassword(int id, String s)`: Encrypts a new password and appends the user ID and the encrypted string as a new line to the text file.
- `protected void updatePassword(int id, String s)`: Reads all stored passwords into memory, finds the specific user ID, replaces their old encrypted password with a newly encrypted one, and overwrites the entire file to save the changes.

D. SystemDataService Class

Methods:

- `public String loadInitialData():`

E. User Class

Fields:

- `private int id`: The unique numeric identifier for the user.
- `private String name`: The user's full name.
- `private String email`: The user's registered email address.
- `private String role`: The user's assigned role or permission level within the system (e.g., admin, standard user).
- `private String status`: The user's current account status (e.g., "ACTIVE").

Methods:

- `public User(int id, String name, String email, String role, String status)`: Constructor that initializes a completely new User object with all required fields.
- `public int getId()`, `public String getName()`, `public String getEmail()`, `public String getRole()`, `public String getStatus()`: Standard getter methods for retrieving private user attributes.
- `public void setName(String name)`, `public void setEmail(String email)`, `public void setRole(String role)`, `public void setStatus(String status)`: Standard setter methods for updating private user attributes.
- `public String toString()`: Overrides the default string representation to return a comma-separated string of the user's attributes.
- `public static User fromString(String line)`: A static factory method that accepts a comma-separated string, parses it based on the delimiter, and returns a newly constructed User object.

F. UserLogin Class

Methods:

- `public void showLogin(UserRepository Repo)` throws `IOException`: Prompts the user for their ID and password via console input. It checks if the user exists using the `UserRepository` and attempts to verify the password against the `PassWordManager`. It enforces a maximum attempt limit (fetched via `ConfigLoader`) and successfully transitions the user to the `DashboardFacade.loadMainPage()` if authentication passes. Throws specific `IOException` messages ("User does not exists" or "Too many attempts") if authentication fails.

G. UserManagementFacade Class

H. UserRegistration Class

Methods:

- `public void showRegistration(UserRepository Repo)` throws `IOException`: Prompts the console for a new user ID. If the ID already exists in the repository, it aborts by throwing an `IOException` ("User already exists"). Otherwise, it collects the new password, securely saves it via the `PassWordManager`, collects the remaining demographic data (email, name, role), instantiates a new `User` object with a default "ACTIVE" status, and persists it to the file system using `UserRepository.addUser()`.

J. UserRepository Class

Fields:

- `private final Path userFilePath`: The designated system file path used for storing user records in a comma-separated text format.

Methods:

- `public UserRepository(String filePath)`: Constructor that initializes the `Path` object and triggers the creation of the underlying file if it is missing.
- `public void createFileIfNotExists()`: Helper method that checks the file system and generates the necessary parent directories and the target text file if they do not already exist.
- `public void addUser(User user)`: Converts a `User` object into its string representation and appends it as a new line to the text file.
- `public List<User> getAllUsers()`: Reads all lines from the text file into memory, skips any blank lines, converts each valid CSV line back into a `User` object via the `fromString` method, and returns the aggregated list.
- `public User findUserById(int id)`: Iterates through the list generated by `getAllUsers()` and returns the specific `User` object matching the provided numeric ID, or null if no match is found.
- `public void updateUser(User updatedUser)`: Retrieves all existing users, locates the one matching the updated user's ID, replaces its text representation in the in-memory list, and truncates/overwrites the entire text file to permanently save the changes.
- `public void deleteUserById(int id)`: Retrieves all existing users, filters out the specific user matching the provided ID, and overwrites the text file with the remaining list of users, effectively deleting the target record.
- `public int countUsers()`: Returns the total number of valid users currently stored in the system.

- `public String getUserData():` An auto-generated stub method currently returning null.
- `public String getName():` An auto-generated stub method currently returning null.

4. Package InviteMangerSubSystem

A. Invite Class

Fields:

- `private int senderId:` The unique numeric identifier of the user sending the invite.
- `private int receiverId:` The unique numeric identifier of the target user receiving the invite.
- `private LocalDateTime expiryDate:` The exact timestamp designating when this invite will expire and become invalid.

Methods:

- `public Invite(int senderId, int receiverId):` Constructor that initializes the sender and receiver IDs. It automatically calculates and sets the `expiryDate` to be exactly 3 months from the exact moment the object is instantiated.
- `public String toString():` Overrides the default string representation to return a comma-separated format (`senderId,receiverId,expiryDate`).

B. InviteManagerFacade Class

Methods:

- `public static String[] getInvites(int id):` Instantiates an `InviteRepository` using paths from the `ConfigLoader`, fetches all raw pending invites, and returns them as an array of strings to be rendered by the dashboard.
- `public static void acceptInvite(String invite):` Processes an accepted invitation. It reads the invites text file into memory, filters out the accepted invite string, and overwrites the file to remove it. Crucially, it then delegates to `FriendRepo.createFriend(invite)` to officially establish the new social connection.
- `public static void rejectInvite(String invite):` Processes a declined invitation. It reads the invites text file into memory, filters out the rejected invite string, and overwrites the file to permanently delete the pending request.

C. InviteRepository Class

Fields:

- `private final Path filePath:` The designated system file path used for storing the invitation records in a text-based format.

Methods:

- `public InviteRepository(String filePath)`: Constructor that initializes the desired file path and invokes the helper method to guarantee the file exists before any read/write operations occur.
- `private void createFileIfNotExists()`: Helper method that safely checks the file system and generates the necessary parent directories and the desired text file if they are missing.
- `public void saveInvite(Invite invite)`: Takes an Invite object, invokes its `toString()` method, converts the resulting string into a byte array, and safely appends it as a new line to the end of the text file.
- `public List<String> getAllRawInvites()`: Opens the desired text file, reads all lines into memory, filters out any accidental blank lines, and returns a list of raw CSV strings representing all stored invitations.

5. Package NotificationSubSystem

A. Notification Class

Fields:

- `public String text`: The actual message or content of the notification.
- `LocalDateTime notifyDateTime`: The timestamp recording when the notification occurred or is scheduled to occur.

Methods:

- `public Notification(String t)`: Constructor that initializes the notification text and automatically assigns the current system time to `notifyDateTime`.
- `public Notification(String t, LocalDateTime time)`: Overloaded constructor that initializes the notification text and allows for setting a specific, custom timestamp instead of the current time.

B. NotificationFacade Class

C. NotificationRepo Class

Fields:

- `private final Path notificationFilePath`: The designated system file path used for storing the notification records in a text-based, comma-separated format.

Methods:

- `public NotificationRepo(String filepath)`: Constructor that initializes the target file path and invokes the helper method to guarantee the file exists before any read/write operations occur.

- `public void createFileIfNotExists():` A utility method that safely checks the file system and generates the necessary parent directories and the target text file if they do not already exist.
- `public void createNotification(String noti, LocalDateTime time):` Formats the provided timestamp (using the strictly defined "yyyy-MM-dd HH:mm:ss" pattern) and appends the notification text and time as a comma-separated new line to the text file. If a null time is passed, it automatically defaults to the current system time.
- `public void deleteNotification(String noti):` Reads all stored notifications into memory, filters out the specific line matching the provided notification text, and truncates/overwrites the text file with the remaining records to effectuate a deletion.
- `public List<Notification> getAllNotifications():` Opens the target text file, reads all lines, skips accidental blank lines, parses the formatted date strings back into standard `LocalDateTime` objects using the designated formatter, and returns a fully populated list of `Notification` objects.

6. Package `ReportDashboardSubSystem`

A. `DashboardFacade` Class

Methods:

- `public static void displayEvent(CompositeEvent compositeEvent):` An overloaded method placeholder intended to trigger the UI rendering logic for a nested, multi-part event.
- `public static void displayEvent(BasicEvent basicEvent):` An overloaded method placeholder intended to trigger the UI rendering logic for a single, standalone event.
- `public static void displayLoginPage():` A placeholder method intended to render the initial authentication screen.
- `public static void loadMainPage():` A placeholder method intended to initialize and load the primary dashboard after login.

B. `EventPage` Class (Implements `Page`)

Fields:

- `private static EventPage instance:` The single, static instance of the `EventPage` class.
- `static List<TextElement> UIElements:` A static list containing the core textual UI components loaded from the configuration file.

Methods:

- `private EventPage():` Private constructor that initializes the `ConfigLoader` and populates the `UIElements` list by reading the specified text file. (*Note: Code currently reads from `Files.mainpage` instead of a specific event page file.*)

- `public static synchronized EventPage getInstance():` Returns the singleton instance of the event page, creating it safely if it does not exist.
- `public void displayPage():` Renders the base UI elements to the console, fetches all existing events via the `EventManagerFacade`, and displays them. It then opens a `Scanner` to capture user input, using a switch statement to route the user to create ('c'), edit ('e'), or return to the main menu ('r').

C. FriendsPage Class (Implements Page)

Fields:

- `private static FriendsPage instance:` The single, static instance of the `FriendsPage` class.
- `static List<TextElement> UIElements:` A static list containing the core textual UI components loaded from the `Files.friendpage` configuration file.

Methods:

- `private FriendsPage():` Private constructor that restricts external instantiation and loads the UI elements.
- `public static synchronized FriendsPage getInstance():` Returns the singleton instance of the friends page.
- `public void displayPage():` Currently an auto-generated stub intended to handle the rendering and interactive console loop for friend management.

D. InvitesPage Class (Implements Page)

Fields:

- `private static InvitesPage instance:` The single, static instance of the `InvitesPage` class.
- `static List<TextElement> UIElements:` A static list containing the core textual UI components loaded from the `Files.invitepage` configuration file.

Methods:

- `private InvitesPage():` Private constructor that restricts external instantiation and loads the UI elements.
- `public static synchronized InvitesPage getInstance():` Returns the singleton instance of the invites page.
- `public void displayPage():` Renders the base UI elements, fetches the currently logged-in user, and retrieves their pending invites. It displays the invites as dynamically created `TextElement` objects. It then opens a console prompt allowing the user to accept the top invite ('y'), reject it ('n'), or return to the main dashboard ('r').

E. MainPage Class (Implements Page)

Fields:

- private static MainPage instance: The single, static instance of the MainPage class.
- static List<TextElement> UIElements: A static list containing the core textual UI components loaded from the Files.mainpage configuration file.

Methods:

- private MainPage(): Private constructor that restricts external instantiation and loads the UI elements.
- public static synchronized MainPage getInstance(): Returns the singleton instance of the main page.
- public void displayPage(): Renders the primary dashboard menu to the console. It opens a Scanner to read numeric user input and uses a switch statement to navigate the user to the EventPage (1), InvitesPage (2), NotificationPage (3), ReportPage (4), or FriendsPage (5). Invalid inputs loop the user back to the current main page.

#	OOP Component	No of times the components have been used
1	Nested Classes	1
2	Abstract Class	1
3	Interface	3
4	Hierarchical Inheritance	Extensive*
5	Package	Extensive*
6	Exception Handling	5
7	I/O: File Handling / Scanner	Extensive*
8	Overloaded Methods	4
9	Overloaded Constructors	2
10	Vararg Overloading	1
11	Wrappers	Extensive*

**Extensive has been used where the said OOP component has been used more than 5 times in the project*